



TECNICATURA SUPERIOR EN

**Ciencia de Datos e
Inteligencia Artificial**

TÉCNICAS DE PROCESAMIENTO DEL HABLA

EJE N°1: Fundamentos y Procesamiento Digital de la
Señal

**Tema: Preparación de Datos Crudos de
Audio**

EJE I: Fundamentos y Procesamiento Digital de la Señal

Tema 2: Preparación de Datos Crudos de Audio

Limpieza, normalización y transformación de archivos de audio como base para el análisis estadístico

Nota para el docente: Las imágenes generadas para este material están identificadas con [ID: X]. Inserte cada imagen en la sección correspondiente según se indica abajo.

OBJETIVOS DE APRENDIZAJE

Al finalizar este módulo, podrás:

- Identificar los formatos de audio más comunes y sus características
- Cargar y explorar archivos de audio crudos usando Python
- Aplicar técnicas básicas de limpieza: recorte de silencios y reducción de ruido
- Normalizar audio en amplitud, duración y frecuencia de muestreo
- Transformar señales para preparar datasets consistentes
- Visualizar datos para diagnóstico antes del análisis estadístico
- Exportar datos procesados listos para EDA y modelado

CONTENIDO INTERACTIVO

SECCIÓN 1: ¿QUÉ SON "DATOS CRUDOS" DE AUDIO?

1.1 Definición y Características

Concepto intuitivo: Los "datos crudos" (raw data) son archivos de audio tal como fueron grabados, sin procesar. Son como una fotografía sin editar: contienen toda la información, pero también ruido, silencios innecesarios y variaciones que dificultan el análisis.

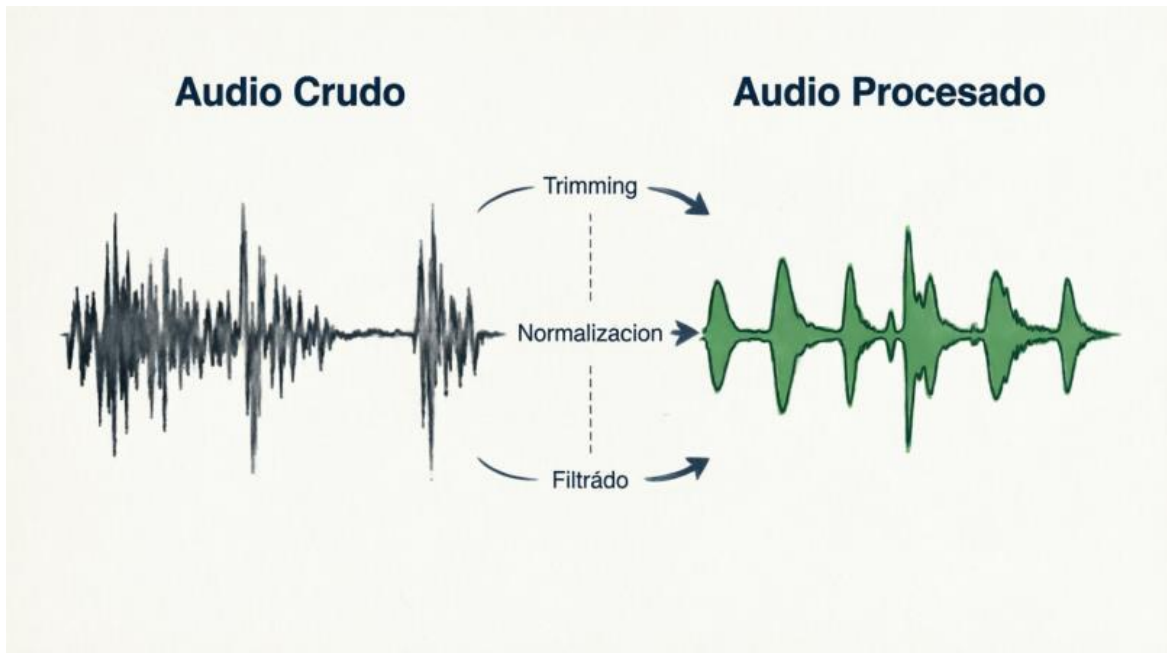


Diagrama comparativo: Audio Crudo vs. Audio Procesado

- Lado izquierdo: onda con ruido, silencios y picos irregulares
- Lado derecho: onda limpia, normalizada y recortada
- Flechas indican los procesos aplicados: trimming, normalización, filtrado

Características típicas de audio crudo:

Característica	Descripción	Impacto en el análisis
Ruido de fondo	Sonidos no deseados (ventilador, tráfico)	Dificulta identificar patrones reales
Silencios largos	Pausas sin información útil	Aumenta tamaño de datos sin valor
Volúmenes variables	Grabaciones a diferentes intensidades	Sesga comparaciones estadísticas
Duraciones distintas	Archivos de 2s a 2 minutos	Imposibilita procesamiento por lotes
Formatos diversos	WAV, MP3, M4A, etc.	Requiere estandarización previa

SECCIÓN 2: FORMATOS DE ARCHIVO DE AUDIO

2.1 Formatos Comunes y sus Propiedades

Clasificación práctica:

SIN COMPRESIÓN (Pérdida cero)

- WAV: Estándar profesional, alta calidad, archivos grandes
- AIFF: Similar a WAV, común en macOS
- FLAC: Compresión sin pérdida, equilibrio calidad/tamaño

CON COMPRESIÓN (Pérdida controlada)

- MP3: Universal, buena compresión, pérdida audible en bajos bitrate
- AAC: Mejor que MP3 a igual bitrate, usado en streaming
- OGG: Open source, buena calidad, menos compatible

Regla de oro para análisis:

Trabaja siempre con el formato de **mayor calidad disponible**. La compresión con pérdida elimina información que podría ser relevante para tu modelo.

SECCIÓN 3: LIMPIEZA DE AUDIO - TRIMMING □

3.1 Recorte de Silencios (Trimming)

Problema: Los silencios al inicio/final no aportan información y desperdician recursos.

Solución intuitiva: Detectar regiones donde la amplitud está por debajo de un umbral y eliminarlas.



Ilustración del proceso de trimming

- Waveform con regiones de silencio marcadas en rojo (< -30 dB)
- Zona central con actividad de voz en verde
- Líneas punteadas mostrando los límites de recorte
- Ejes: Tiempo (s) y Amplitud

Parámetros clave:

- `top_db`: Umbral en decibeles (típico: 20-40 dB)
- Valores más altos → recorte más agresivo
- Valores más bajos → conserva más audio (pero también más ruido)

Código conceptual:

```
import librosa  
  
# Elimina silencios por debajo de -30 dB respecto al pico  
audio_limpio, _ = librosa.effects.trim(audio, top_db=30)
```

SECCIÓN 4: NORMALIZACIÓN DE AMPLITUD

4.1 ¿Por qué Normalizar el Volumen?

Analogía: Imagina que comparas alturas de personas, pero algunas están en metros y otras en pies. ¡Necesitas una unidad común! Lo mismo con audio: diferentes volúmenes sesgan el análisis de energía.

Diagrama de normalización de amplitud:

- Gráfica superior: señal original con picos en +0.6 y -0.4
- Fórmula central: $\text{audio_norm} = \text{audio} / \max(|\text{audio}|)$
- Gráfica inferior: misma onda escalada con picos en +1.0 y -1.0
- Ejes claros y colores profesionales

Fórmula explicada:

$$\text{audio_norm} = \text{audio} / \max(|\text{audio}|)$$

Donde:

- $\max(|\text{audio}|)$ = valor absoluto máximo de la señal
- Resultado: pico más alto = ± 1.0 , resto proporcional

Código práctico:

```
# Normalización peak: escalar al rango [-1, +1]
if np.max(np.abs(audio)) > 0:
    audio_normalizado = audio / np.max(np.abs(audio))
```


SECCIÓN 5: NORMALIZACIÓN DE SAMPLE RATE

5.1 Unificando la Frecuencia de Muestreo

Problema: Algunos archivos a 22050 Hz, otros a 44100 Hz → incompatibles para procesamiento por lotes.

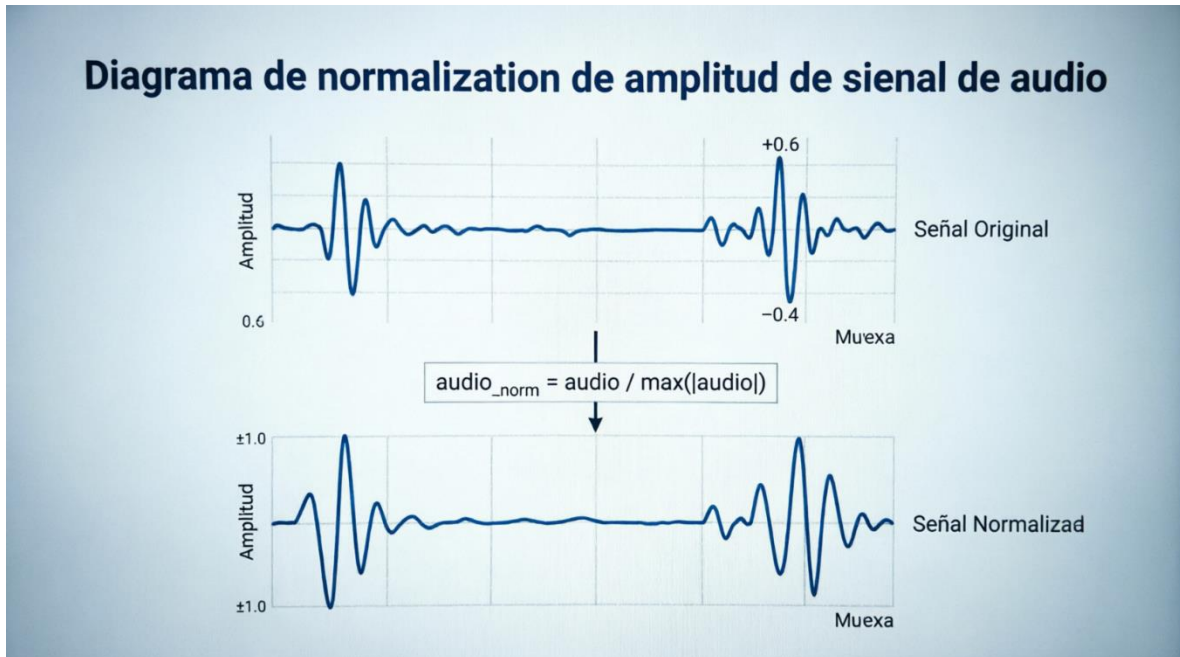


Ilustración de remuestreo (resampling)

- Parte superior: señal con alta densidad de puntos (44100 Hz)
- Parte inferior: misma señal con menor densidad (22050 Hz)
- Nota: "Misma información, menos datos"
- Estilo técnico con colores azul y naranja

Solución: Remuestrear todos a una frecuencia común recomendada para análisis de voz.

python

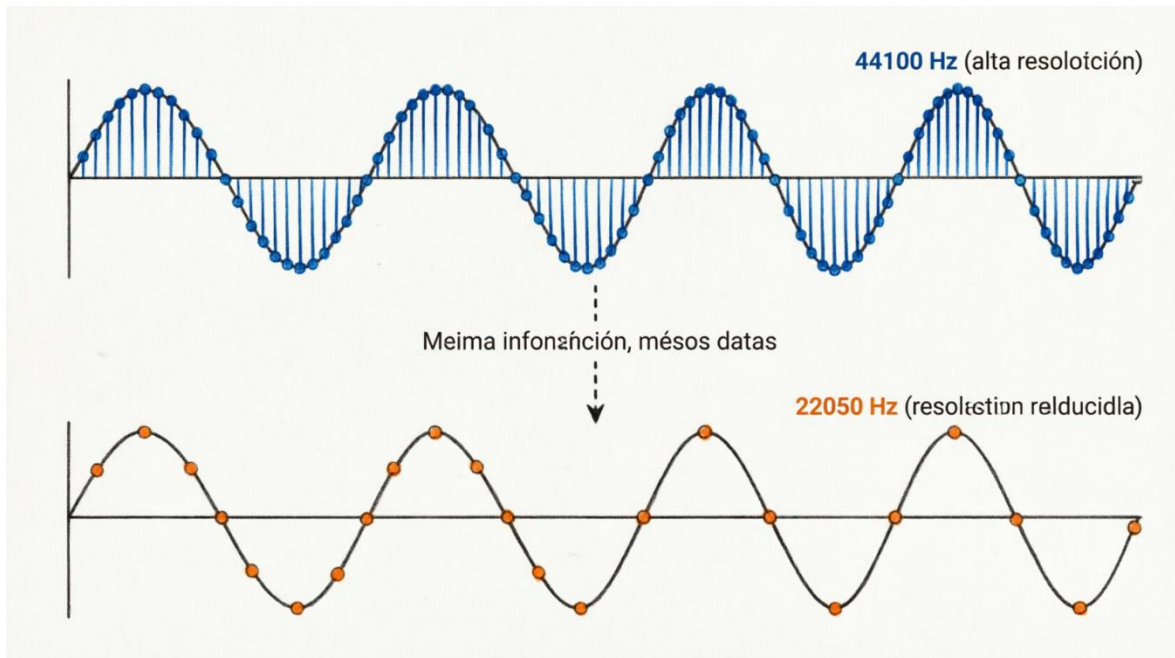
```
# Convertir a 22050 Hz (estándar para análisis de voz)
audio_resampleado = librosa.resample(audio, orig_sr=sr, target_sr=22050)
```

¿Por qué 22050 Hz?

- Cubre el rango de voz humana (80 Hz - 8000 Hz) con margen de Nyquist
- Reduce el tamaño de datos a la mitad vs. 44100 Hz
- Es suficiente para la mayoría de tareas de clasificación de audio

SECCIÓN 6: VISUALIZACIÓN PARA DIAGNÓSTICO

6.1 Waveform + Espectrograma: Vista Dual



Visualización combinada tiempo-frecuencia

- Panel superior: waveform (amplitud vs. tiempo)
- Panel inferior: espectrograma (frecuencia vs. tiempo, color = intensidad)
- Bandas brillantes en 300 Hz y 500 Hz muestran frecuencias presentes
- Títulos y ejes claramente etiquetados en español

Interpretación guiada:

Visualización	¿Qué revela?	¿Cuándo usarla?
Waveform	Patrones de energía, silencios, picos, duración	Diagnóstico rápido de calidad
Espectrograma	Frecuencias presentes, evolución temporal, armónicos	Identificar contenido espectral

Código de visualización:

python

```
import matplotlib.pyplot as plt
import librosa.display

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 5))

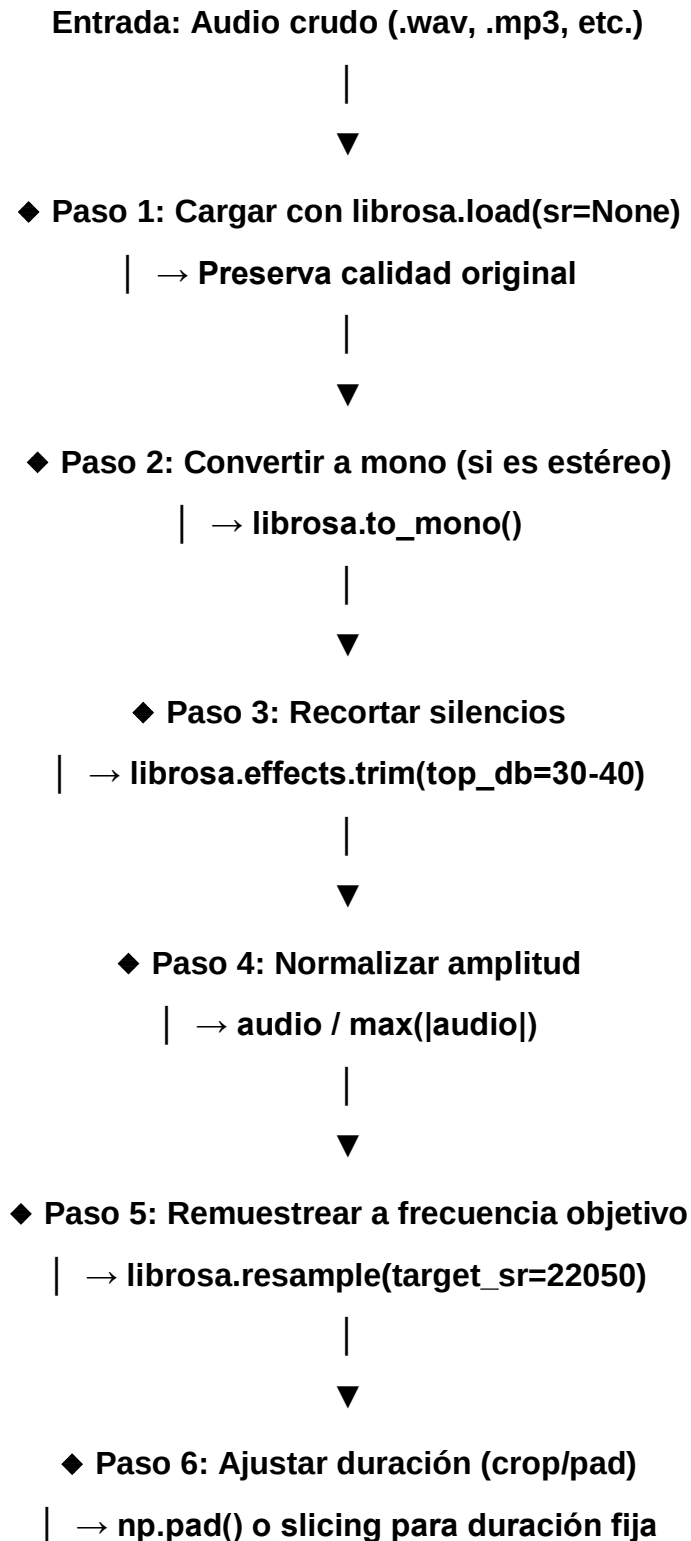
# Waveform
librosa.display.waveshow(audio, sr=sr, ax=ax1)
ax1.set_title('Waveform: Amplitud vs. Tiempo')

# Espectrograma
D = np.abs(librosa.stft(audio))
S_db = librosa.amplitude_to_db(D, ref=np.max)
librosa.display.specshow(S_db, sr=sr, x_axis='time', y_axis='hz', ax=ax2,
cmap='magma')
ax2.set_title('Espectrograma (dB)')

plt.tight_layout()
plt.show()
```

SECCIÓN 7: PIPELINE COMPLETO DE PREPROCESAMIENTO ∞

7.1 Flujo Recomendado Paso a Paso





◆ Paso 7: Validar visualmente

| → waveform + espectrograma rápido



⬆ Salida: Audio limpio, normalizado y listo para EDA

7.2 Función Reutilizable

python

```
def preparar_audio_para_analisis(ruta_archivo,
                                target_sr=22050,
                                target_dur=3.0,
                                top_db=35):
    """
    Pipeline completo de preprocessing para análisis de audio.

    Retorna: audio procesado, sample rate, diccionario de metadatos
    """
    import librosa
    import numpy as np

    # 1. Cargar
    audio, sr = librosa.load(ruta_archivo, sr=None)

    # 2. Mono
    if audio.ndim == 2:
        audio = librosa.to_mono(audio)

    # 3. Trim silencios
    audio, _ = librosa.effects.trim(audio, top_db=top_db)

    # 4. Normalizar amplitud
    if np.max(np.abs(audio)) > 0:
        audio = audio / np.max(np.abs(audio))

    # 5. Resamplear
    if sr != target_sr:
```

```
audio = librosa.resample(audio, orig_sr=sr, target_sr=target_sr)
sr = target_sr

# 6. Ajustar duración
samples_obj = int(target_dur * sr)
if len(audio) > samples_obj:
    inicio = (len(audio) - samples_obj) // 2
    audio = audio[inicio:inicio + samples_obj]
else:
    audio = np.pad(audio, (0, samples_obj - len(audio)), mode='constant')

# 7. Metadatos
metadatos = {
    'duracion': target_dur,
    'sample_rate': sr,
    'rms': np.sqrt(np.mean(audio**2)),
    'zcr': librosa.feature.zero_crossing_rate(audio).mean()
}

return audio, sr, metadatos
```

SECCIÓN 8: EXTRACCIÓN DE FEATURES BÁSICAS PARA EDA

8.1 Métricas Descriptivas Iniciales

Antes de técnicas avanzadas como MFCC, extrae características simples para análisis exploratorio:

Feature	Qué mide	Interpretación
Energía RMS	Potencia promedio de la señal	¿Qué tan "fuerte" es el audio en general?
Zero-Crossing Rate	Frecuencia de cambios de signo (+/-)	¿Es una señal "suave" o "áspera"?
Spectral Centroid	"Centro de gravedad" frecuencial	¿El sonido es "grave" o "agudo"?
Spectral Rolloff	Frecuencia que acumula 85% de energía	¿Dónde se concentra la energía espectral?

Código de extracción:

python

```
def extraer_features_eda(audio, sr):
    """Extrae features básicas para análisis exploratorio"""
    import librosa
    import numpy as np

    return {
        'rms_mean': librosa.feature.rms(y=audio)[0].mean(),
        'zcr_mean': librosa.feature.zero_crossing_rate(audio)[0].mean(),
        'centroid_mean': librosa.feature.spectral_centroid(y=audio, sr=sr)[0].mean(),
        'rolloff_mean': librosa.feature.spectral_rolloff(y=audio, sr=sr)[0].mean(),
        'duracion': len(audio) / sr
    }

# Uso
features = extraer_features_eda(audio_procesado, sr)
print(features)
# Ejemplo salida: {'rms_mean': 0.23, 'zcr_mean': 0.045, ...}
```


ACTIVIDADES INTERACTIVAS (OPTATIVAS)

Actividad 1: Clasificador de Formatos 🎧

Escenario: Tienes 10 archivos con extensiones mixtas (.wav, .mp3, .m4a).

Tarea:

1. Investiga: ¿Cuál formato preserva más información para análisis?
2. Propón: ¿En qué orden convertirías los archivos para estandarizar?
3. Justifica: ¿Por qué no trabajar directamente con MP3 comprimido?

Respuesta guía:

1. WAV/FLAC > MP3 (sin pérdida vs con pérdida)
2. Orden: MP3→WAV (descomprimir) → procesar → exportar a FLAC (archivar)
3. MP3 elimina frecuencias "menos perceptibles" que podrían ser relevantes para un modelo de ML (ej. armónicos sutiles).

Actividad 2: Umbral de Trimming 🗂️

Objetivo: Experimentar mentalmente con el parámetro top_db.

Preguntas:

- Si usas top_db=10, ¿qué esperas que ocurra?
- Si usas top_db=60, ¿qué riesgo corres?
- Para voz en entorno ruidoso, ¿qué valor intermedio probarías primero?

Respuesta guía:

- top_db=10: Recorte muy agresivo → puede eliminar partes suaves de la voz
- top_db=60: Recorte muy suave → deja mucho ruido de fondo
- Valor inicial recomendado: 30-40 dB (equilibrio voz/ruido)

RESUMEN DE CONCEPTOS CLAVE

Concepto	Propósito	Herramienta clave	Parámetro típico
Trimming	Eliminar silencios inútiles	librosa.effects.trim()	top_db=30-40
Normalización de amplitud	Estandarizar volumen	audio np.max(np.abs(audio))	Rango [-1, +1]
Remuestreo	Unificar sample rate	librosa.resample()	target_sr=22050
Ajuste de duración	Entradas de tamaño fijo	np.pad() / slicing	duración_objetivo
Conversión mono	Simplificar análisis	librosa.to_mono()	-
Diagnóstico visual	Validar calidad	librosa.display.waveshow()	-

EVALUACIÓN FORMATIVA (OPTATIVA)

Preguntas de Verificación:

1. Nivel Básico:

- ¿Por qué es importante trabajar con audio sin compresión para análisis?
- Nombra dos problemas comunes en datos crudos de audio.
- ¿Qué hace la normalización de amplitud?

2. Nivel Intermedio:

- Explica la diferencia entre recortar (crop) y rellenar (pad) para ajustar duración.
- ¿Por qué convertir a mono antes de extraer características?
- Si un audio tiene sample rate de 44100 Hz y lo remuestras a 22050 Hz, ¿qué ocurre con la resolución temporal?

3. Nivel Avanzado:

- Diseña un criterio automático para detectar si un audio está saturado (clipping).
- Propón una estrategia para manejar un dataset con duraciones entre 0.5s y 10s, cuando tu modelo requiere entradas de 3.0s.
- ¿Cómo usarías el espectrograma para decidir si un audio necesita filtrado de ruido?

RECURSOS ADICIONALES

Documentación Oficial:

- [Librosa: Audio Loading](#)
- [Librosa: Effects](#)
- [Librosa: Display](#)

Repositorios Recomendados (de tu KB):

- [huggingface/transformers](#) - Modelos pre-entrenados para audio y texto
- [explosion/spaCy](#) - Procesamiento de lenguaje natural industrial
- [NVIDIA/DeepLearningExamples](#) - Ejemplos de DL incluyendo ASR
- [Speech and Language Processing - Jurafsky](#) - Libro de referencia gratuito

Herramientas Complementarias:

- **Audacity**: Editor gráfico para validación manual
- **SoX**: Procesamiento por línea de comandos (batch)
- **FFmpeg**: Conversión masiva de formatos

GLOSARIO

Término	Definición
Raw audio	Audio sin procesar, tal como fue grabado
Trimming	Eliminación automática de silencios en bordes
Peak normalization	Escalar señal para que su pico máximo sea ± 1.0
Resampling	Cambiar la frecuencia de muestreo de una señal
Hop length	Desplazamiento entre ventanas en segmentación
RMS	Root Mean Square: medida de energía promedio
Zero-Crossing	Puntos donde la señal cambia de signo (+ a -)
Pipeline	Secuencia ordenada de transformaciones aplicadas

PRÓXIMOS PASOS

Con los datos limpios y normalizados, avanzaremos al **EJE I - Tema 3**:

- **Análisis Exploratorio de Audio (EDA)**: Identificación de variables y aplicación práctica de la Transformada de Fourier
- Introducción a características espectrales y su interpretación estadística

Checklist de salida del Tema 2:

- ☐ Entiendo por qué la calidad del formato importa
- ☐ Puedo cargar y explorar un audio con librosa
- ☐ Sé aplicar trimming, normalización y remuestreo
- ☐ Puedo diagnosticar calidad con waveform y espectrograma
- ☐ Tengo un pipeline reproducible de preprocessing

¿Preguntas? Recuerda: *Garbage in, garbage out*. La calidad de tu análisis depende directamente de la preparación de tus datos. ¡Invierte tiempo aquí para ahorrar problemas después!

Material desarrollado para: Procesamiento Digital de Señales de Audio

Nivel: Introductorio-Intermedio

Duración estimada: 3-4 horas de estudio + práctica